

TransitFlow Graph Database Design

Rationale說明

Section 3 — Graph Database Design Rationale

1. Data Storage Strategy: Nodes, Relationships, and Properties

在我們的 Neo4j 設計中，資料被嚴格劃分為節點、關聯與屬性，以最大化圖形演算法的執行效能與語意清晰度：

- **Nodes**: 我們將實體車站儲存為節點，並賦予具體的標籤 `:MetroStation` 與 `:NationalRailStation`。
 - **Justification**: 車站是路網拓樸的頂點。捨棄通用的 `:Station` 標籤而採用具體雙標籤，能在執行單一網路查詢時達到 $O(1)$ 的標籤過濾效能，避免掃描無關網路的節點。
- **Relationships**: 我們建立了 `:METRO_LINK`、`:RAIL_LINK` 以及跨網路的 `:INTERCHANGE_TO` 關聯線。
 - **Justification**: 關聯線代表了路網的邊。將路線定義為實體關聯而非屬性，能讓我們利用 Neo4j 的免索引鄰接特性。這表示資料庫在尋訪相鄰車站時，是直接透過記憶體指標進行跳躍，完全免除了關聯式資料庫中昂貴的 JOIN 查表操作。
- **Properties**: 節點儲存地理與狀態屬性（如 `zone`, `closed`）；關聯線儲存成本權重（如 `travel_time_min`, `fare_standard`, `fare_first`）。
 - **Justification**: 時間與金錢是依存於兩站之間的權重成本，將其存於邊上，能讓尋徑演算法在遍歷時動態累加。將 `closed` 狀態存於節點，則讓我們能在不破壞圖形拓樸的前提下，實現即時的故障避障邏輯。

2. Graph vs. Relational: The Algorithmic Argument

針對交通系統的路由情境（如 Shortest Path 與 Delay Ripple），圖形資料庫在底層演算法層面具備 SQL 無法比擬的壓倒性優勢：

- **The Relational (SQL) Bottleneck**: 若在 PostgreSQL 中尋找未知深度的最短路徑或漣漪效應，必須使用遞迴通用資料表運算式。這在演算法上會引發大量的 Self-Joins 與 Table Scans，隨著搜尋深度增加，會產生嚴重的笛卡爾積問題，時間複雜度呈指數級惡化。
- **The Graph Advantage**: Neo4j 原生支援圖形遍歷。
 - 針對最短路徑：我們能直接套用 Dijkstra 演算法的概念（時間複雜度為 $O(V \log V + E)$ ），透過累加邊的權重高效找到最佳解。
 - 針對延誤漣漪 (`query_delay_ripple`): 我們使用變動長度路徑 `*0..N`，這在底層是一個優化過的廣度優先搜尋。圖形資料庫只需尋訪相鄰的指標，使得時間複雜度僅與受影響的局部子圖大小有關，而非整個資料庫的總資料量。

3. Enabling Specific Query Types via Graph Model

我們設計的圖形結構支援並簡化了以下複雜查詢的實作：

Query 1: Shortest Route (`query_shortest_route`)

- **How the model enables it:** 由於我們將時間成本存在邊的屬性 `travel_time_min` 上, 且使用原生圖形遍歷, 我們可以直接在 Cypher 中利用 `reduce()` 函數動態加總整條路徑的時間, 並搭配 `ORDER BY` 模擬 Dijkstra 演算法找出時間最短的最佳解。同時, 因為節點上獨立設計了 `closed` 狀態屬性, 我們能在硬體尋徑的過程中, 透過 `NONE()` 條件即時避開故障車站, 不需在應用層寫複雜的過濾邏輯。

Query 2: Cross-Network Interchange (`query_interchange_path`)

- **How the model enables it:** 由於我們將跨網路通道獨立設計為一種專屬的關聯型態 (`:INTERCHANGE_TO`), 我們在 Cypher 中只需宣告 `WHERE ANY(r IN relationships(path) WHERE type(r) = 'INTERCHANGE_TO')`。這種明確的 Schema 設計, 讓資料庫能在圖形拓撲配對階段就直接篩選出轉乘路徑。

Query 3: Cheapest Route with Dynamic Fares (`query_cheapest_route`)

- **How the model enables it:** 此查詢需要依據艙等與跨區幅度動態計算票價。因為我們將成本屬性 (`fare_standard`) 放在邊上, 並將分區 (`zone`) 放在節點上, 我們能直接在圖形遍歷的階段累加邊的成本, 並計算 `max(zone) - min(zone)` 的附加費。Node 與 Edge 屬性分離的設計, 讓計價邏輯變得極為直觀。

4. Node Identity

- **Unique Identifier:** 我們選擇真實世界的站點代碼 `station_id` (例如: "MS01", "NR01") 作為節點的身分證。
- **Justification for choice:**
 1. **Determinism:** `station_id` 是一個具有自然商業邏輯的唯一識別碼。使用它取代資料庫自動生成的 UUID 或整數 ID, 能確保資料匯入時透過 `MERGE` 達成絕對的幂等性, 避免重複寫入。
 2. **Index Optimization:** 我們在資料庫初始化時, 利用 `CREATE CONSTRAINT ... REQUIRE s.station_id IS UNIQUE` 強制其唯一性。這在底層會自動生成 B-Tree 索引, 讓所有路由查詢在尋找起點與終點時的定位時間從 $O(N)$ 降至 $O(\log N)$ 。
 3. **System Integration:** 此 ID 可與 PostgreSQL 關聯式資料庫中的 `stations` 表格無縫對接, 作為跨資料庫的虛擬 Foreign Key。



(Figure 1: Visualisation of the generated graph topology showing stations and interconnecting links.)